Proposition 20 (Conditional recovery deadline)**.** *Consider this serial schedule: a nonce-bound checkpoint in one round trip T_{rt}; package fetch in one T_{rt}; the request committed and delivered to participating honest parties within one log call $T_{\log}$; approvals available within T_{app} of request observation; the approval certificate committed within one further $T_{\log}$; a bound channel in one T_{rt}; responses in one T_{rt}; local work T_{loc}. Assume clock rates in $[1-\rho, 1+\rho]$ with $0 \le \rho < 1$, and that each guardian's hold of $(1+\rho)\tau$ local units starts when it observes the committed request. Then*

$$\begin{aligned} T_{\mathrm{restore}} &\le 4T_{\mathrm{rt}} + T_{\log} + T_{\mathrm{loc}} + \max\Big\{T_{\mathrm{app}} + T_{\log},\ \tfrac{1+\rho}{1-\rho}\tau\Big\} \\ &\le 4T_{\mathrm{rt}} + 2T_{\log} + T_{\mathrm{loc}} + \max\Big\{T_{\mathrm{app}},\ \tfrac{1+\rho}{1-\rho}\tau\Big\}. \end{aligned} \tag{5}$$

If instead the hold starts at certificate observation, the bound is $4T_{\mathrm{rt}} + 2T_{\log} + T_{\mathrm{app}} + \frac{1+\rho}{1-\rho}\tau + T_{\mathrm{loc}}$. If the checkpoint and package fetches run concurrently, one T_{rt} can be removed from each bound.

Proof A local wait of ℓ units at rate $r \in [1-\rho, 1+\rho]$ lasts ℓ/r real units, which lies in $[\tau, (1+\rho)\tau/(1-\rho)]$ for $\ell = (1+\rho)\tau$. After the request is observed, release needs both a committed certificate (by $T_{\mathrm{app}} + T_{\log}$) and an expired hold (by $(1+\rho)\tau/(1-\rho)$); these run concurrently. The other stages are serial. The second inequality holds because $\max(T_{\mathrm{app}} + T_{\log}, x) \le \max(T_{\mathrm{app}}, x) + T_{\log}$. □

The bound depends on that hold-start assumption. $T_{\mathrm{rt}}, T_{\log}, T_{\mathrm{app}}, T_{\mathrm{loc}}$ are assumptions, not measured latencies, and permanent suppression yields failure, never timeout-created approval.

8 Cryptographic profile and component security

8.1 Profile

Independent 32-byte epoch seeds are embedded in the scalar field of NIST P-384. The sharing prime is the group order (hexadecimal; concatenate the lines):

```
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFC7634D81F4372DDF
581A0DB248B0A77AECEC196ACCC52973
```

Scalars use 48-byte big-endian encodings strictly below that prime; coefficients and blinders are uniform including zero; indices are distinct nonzero elements with $n \le 255$. The second generator uses RFC 9380 suite `P384_XMD:SHA-384_SSWU_RO_`, message `h`, domain tag `CEAB-v2-Pedersen-H`, retrying with an encoded counter for identity or $\pm g_0$ [34, 35]. That retry guarantees $\beta \neq 0$, which Theorem 14 needs; the unknown discrete-log relation is an assumption. The label "v2 in the domain tags is a fixed wire constant.

A capsule carries share and blinder, 96 raw bytes. HPKE info binds account, configuration, bank, epoch, generation and holder; associated data additionally binds

Table 7 Concrete algorithm choices. Specifying the profile is separate from proving its composition (Theorem 25).

Component	Profile and binding requirements
Capsule PKE	RFC 9180 HPKE Base; DHKEM(X25519, HKDF-SHA256) KEM 0x0020; KDF 0x0001; AEAD 0x0003 (ChaCha20Poly1305). Independent finite-bank slots; signed manifest supplies sender/context authentication [29]
Object PRF	HMAC-SHA256 keyed by the 32-byte epoch seed over typed deterministic CBOR (domain, account, configuration, birth epoch, object ID, immutable header)
Data AEAD	ChaCha20-Poly1305; associated data CBOR(H(ctx), i, *final*) with a final-chunk flag (Section 8.3); nonce is four zero bytes then the uint64 big-endian chunk index, never repeated under an object key; at most $2^{32} - 1$ chunks of 1 MiB [36]
Signatures	Pure Ed25519 on SHA-256 of a typed, domain-separated encoding; canonical validating verification; not Ed25519ph [37]
Encoding	SHA-256; RFC 8949 core deterministic CBOR; reject duplicates, unsupported required fields, invalid scalars/points and oversized requests [38]
Private release	TLS 1.3 with ephemeral exchange, exporter binding to recipient and request, no early-data release, explicit session/buffer cleanup [39]

commitments, package identity and algorithms. Random 32-byte object identifiers collide with probability at most $q(q-1)/2^{257}$. The profile targets about 128-bit classical strength, bounded by X25519 and the 128-bit AEAD/PRF components; a 256-bit seed is not a 256-bit system claim.

8.2 Adaptive exposure game and restricted-adaptive composition

Definition 21 (Record game under adaptive exposure)**.** Fix the policy: N recipient slots, and for the target epoch generations $g = 1, \dots, G$ with n_g holders, threshold t_g and delivery map $\delta(g, x)$ to slots. Setup is honest. It generates every slot key pair, every signing key and every epoch seed, the target seed S uniform on $\{0,1\}^{256}$, and the Pedersen generators, and publishes the directory. The adversary $\mathcal{A}$ may call, in any order and adaptively:

1. $\mathsf{Publish}(e, g)$: an honest generation of S_e, returning commitments and HPKE capsules, each addressed to its slot;
2. $\mathsf{Expose}(s)$: the secret key of slot s; device compromise and bank copying are sets of such calls, and bank copying includes future slots;
3. $\mathsf{Disclose}(e, g, x, \alpha, \gamma)$: the coordinate $\alpha A_g(x) + \gamma B_g(x)$, which covers responses, buffers and partial disclosures;
4. $\mathsf{Dec}(s, c, \mathit{info})$: HPKE decryption under slot s, except on a capsule that $\mathsf{Publish}$ produced for s with that *info*;
5. $\mathsf{SeedRoute}(e)$: S_e, which covers recoverer and archive compromise;
6. $\mathsf{Store}(e, m)$: the client samples a fresh uniform 32-byte identifier, forms the object context (domain, account, configuration, birth epoch, identifier, immutable header), derives the object key HMAC(S_e, ctx) and encrypts the object in chunks, one AEAD call per chunk; an interrupted creation is abandoned and retried under a fresh identifier, which is a new Store;

7. $\mathsf{Open}(e, ctx, (c_1, \ldots, c_\ell))$: one object-level request naming one context. The client derives the key for ctx once, memoized per context, and makes one AEAD decryption call per supplied chunk, including calls that fail authentication;
8. $\mathsf{Identity}(i)$: a signing key, independent of all encryption material;
9. $\mathsf{Challenge}(m_0, m_1)$, at most once (later calls are ignored), for a target-epoch record of equal-length plaintexts, answered with the encryption of m_b.

For a *published* target generation g, the touched set I_g contains index x iff (i) slot $\delta(g, x)$ is exposed at any time, before or after publication, or (ii) some disclosed coordinate at (g, x) is not a multiple of $(1, \beta)$. A slot reused by several generations therefore touches the corresponding index in each of them. Unpublished generations impose no condition. The run is *fresh* if no $\mathsf{SeedRoute}$ touched the target epoch, Open is not applied to the challenge, and $|I_g| < t_g$ for every published target generation. A fresh run outputs $[b' = b]$; any other run outputs a fair bit, and $\mathrm{Adv} = |\Pr[\mathsf{out} = 1] - \frac{1}{2}|$. Each non-freshness event is monotone: once it occurs it persists. Counting a partial disclosure as touching is conservative (Section 6, scope of the exponent view).

A target capsule (g, x) is *late-exposed* if its slot is exposed after the capsule was published. A slot exposed before publication, including a future bank slot, makes the capsule real but not late-exposed. Let Λ_g be the set of late-exposed indices. On a fresh run $\Lambda_g \subseteq I_g$, so $|\Lambda_g| < t_g$. The guessing space is $\mathcal{P} = \prod_g \{\Lambda_g \subseteq [n_g] : |\Lambda_g| < t_g\}$, of size

$$L = |\mathcal{P}| = \prod_{g=1}^{G} \sum_{j=0}^{t_g - 1} \binom{n_g}{j}. \tag{6}$$

$\mathcal{P}$ contains every pattern a fresh run can produce, but it is a conservative superset: under a delivery map that reuses slots, exposures are correlated and fewer patterns are reachable (Section 10). Exposure is *forward-ordered* if no target capsule is late-exposed; then $\mathcal{P}$ can be taken as $\{\varnothing\}$ and $L = 1$. Copying a bank before the generations it will receive are published is forward-ordered; copying it afterwards is not.

Definition 22 (Assumptions and advantage conventions)**.** Every primitive advantage below is a *distinguishing* advantage, $|\Pr[\mathcal{A}'^{\mathrm{real}} \Rightarrow 1] - \Pr[\mathcal{A}'^{\mathrm{ideal}} \Rightarrow 1]|$ for a distinguisher $\mathcal{A}'$. The record game alone uses the bit-guessing form $|\Pr[\mathsf{out} = 1] - \frac{1}{2}|$. A primitive bound stated in bit-guessing form ε' must be doubled before use, because $\varepsilon = 2\varepsilon'$. $\varepsilon_{\mathrm{cca}}$ is the distinguishing advantage against labeled real-or-zero IND-CCA security of HPKE Base mode, for one challenge ciphertext under one key, with label $(info, aad)$ [29]; we take it as an assumption. $\varepsilon_{\mathrm{mu}}(N, M)$ is the advantage in the multi-user version with N keys, M real-or-zero challenge queries, decryption queries except on a challenge ciphertext under its own key and label (the same ciphertext under another label may be decrypted), and adaptive corruption of keys that have received no challenge; a corrupted key may not be challenged. $\varepsilon_{\mathrm{prf}}(Q)$ is the advantage of an adversary making Q adaptive queries against HMAC-SHA256 as a PRF keyed by a uniform 32-byte key; repeated queries return consistent answers by definition. $\varepsilon_{\mathrm{ae}}^{\mathrm{iid}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}})$ is the multi-key all-in-one AE advantage of ChaCha20-Poly1305. The game samples

K_{key} key *instances* independently and uniformly, with replacement, so two instances may hold equal bytes, and some instances may be used only for decryption. It allows C chunk-level encryption calls in total, with nonces never repeated under the same instance index, and D_{dec} chunk-level decryption calls on ciphertexts that are not outputs of encryption calls under the same instance and nonce, including calls that fail authentication. $\varepsilon_{\text{ae}}^{\text{dist}}(K_{\text{key}}, C, D_{\text{dec}})$ is the same with K_{key} distinct keys, sampled uniformly without replacement. Multi-user bounds for ChaCha20-Poly1305 are given in [40]. We could not inspect that paper's key-sampling convention, so Lemma 24 relates the two forms. All advantages are over adversaries running in time close to $\mathcal{A}$'s.

Lemma 23 (Multi-user corruption)**.** $\varepsilon_{\text{mu}}(N, M) \le N\,M\,\varepsilon_{\text{cca}}$*. The factor* NM *is the loss of this generic reduction, not a tight bound; scheme-specific analyses may do better.*

Proof In the multi-user game the adversary $\mathcal{A}'$ makes at most M challenge queries $(i, m_0, m_1, \mathit{info})$, decryption queries (i, c, info) other than on challenge outputs, and corruptions of keys that have received no challenge. Order the challenge queries by time. Hybrid H_j $(0 \le j \le M)$ answers the first j challenge queries with encryptions of m_1 (zeros) and the remaining ones with encryptions of m_0, so H_0 and H_M are the two worlds. Fix $j \ge 1$; Table 8 specifies the reduction B_j.

Table 8 The reduction B_j for the hop $H_{j-1} \to H_j$ in Lemma 23.

Item	Description
Games	H_{j-1} and H_j differ only in the answer to the j-th challenge query: m_0 versus m_1.
Reduction	Sample $i^* \in [N]$ uniformly. Use the single-key challenger's public key at i^* and generate the other $N-1$ key pairs. Answer challenge queries before the j-th with self-made encryptions of m_1, and later ones with self-made encryptions of m_0. Forward the j-th query to the single-key challenge oracle if it is under i^*. Answer decryption under i^* with the single-key oracle, and decryption under other keys with the known secret keys. Answer corruption of $i \ne i^*$ directly.
Query bound	One challenge query; at most as many decryption queries as $\mathcal{A}'$ makes under i^*.
Abort condition	Output a fair bit if the j-th query is not under i^*, or if $\mathcal{A}'$ corrupts i^*, which can then never receive the j-th query. If there is no j-th query, H_{j-1} and H_j coincide and B_j outputs $\mathcal{A}'$'s output.
Freshness	Decryptions of B_j's own challenge outputs are excluded by the multi-user game. The single-key oracle refuses only the forwarded challenge, which is also excluded for $\mathcal{A}'$. Corruption of a challenged key is not allowed to $\mathcal{A}'$, so B_j never has to reveal the embedded key after the j-th query.
Advantage	Compare B_j with the hybrid execution it imitates. The index i^* is independent of that execution, so on runs with a j-th query the query is under i^* with probability exactly $1/N$. On those runs i^* was never corrupted (a corrupted key cannot be challenged), so B_j simulates H_{j-1} or H_j perfectly to the end. On all other runs its output does not depend on the challenge bit. Hence $\varepsilon_{\text{cca}} \ge \frac{1}{N}\lvert \Pr[H_j = 1] - \Pr[H_{j-1} = 1]\rvert$.

Summing the M hops gives the lemma. Keys, capsules and generations are not counted separately: N counts keys and M counts challenge queries, and Theorem 25 makes at most one challenge query per eligible capsule. □

Lemma 24 (Key instances versus key bytes)**.** *For all* $K_{\mathrm{key}}, C, D_{\mathrm{dec}}$*,*

$$\varepsilon^{\mathrm{iid}}_{\mathrm{ae}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}}) \le \varepsilon^{\mathrm{dist}}_{\mathrm{ae}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}}) + K_{\mathrm{key}}(K_{\mathrm{key}} - 1)/2^{257}.$$

Proof Fix a distinguisher $\mathcal{A}'$ and write $p^{\mathrm{iid}}_{\mathrm{real}}$, $p^{\mathrm{dist}}_{\mathrm{real}}$ and p_{ideal} for its probabilities of outputting 1. The ideal world answers encryptions with fresh uniform strings and decryptions with $\perp$, per instance index. Its behaviour therefore does not depend on the key values, and it is the same distribution in both games. Let Col be the event that two of the K_{key} i.i.d. instances are equal, with probability $\delta = 1 - \prod_{i<K_{\mathrm{key}}}(1 - i/2^{256}) \le K_{\mathrm{key}}(K_{\mathrm{key}} - 1)/2^{257}$. Conditioned on $\neg$Col, the i.i.d. tuple is uniform over tuples of distinct keys, which is exactly the distinct-key game. Writing $p^{\mathrm{col}}_{\mathrm{real}}$ for the real-world probability conditioned on Col, the common mixture gives

$$p^{\mathrm{iid}}_{\mathrm{real}} - p_{\mathrm{ideal}} = (1-\delta)\big(p^{\mathrm{dist}}_{\mathrm{real}} - p_{\mathrm{ideal}}\big) + \delta\big(p^{\mathrm{col}}_{\mathrm{real}} - p_{\mathrm{ideal}}\big),$$

where p_{ideal} is split by the same weights because it does not depend on the keys. The last bracket has absolute value at most 1, so $|p^{\mathrm{iid}}_{\mathrm{real}} - p_{\mathrm{ideal}}| \le (1-\delta)\,|p^{\mathrm{dist}}_{\mathrm{real}} - p_{\mathrm{ideal}}| + \delta \le \varepsilon^{\mathrm{dist}}_{\mathrm{ae}} + \delta$. Taking the supremum over $\mathcal{A}'$ proves the lemma. Bounding the real and ideal probabilities separately would have given 2δ; the mixture avoids that. In the i.i.d. game a coincidence lets two instances, each nonce-respecting by index, use the same key bytes with the same nonce (chunk index 0). That event is part of $\varepsilon^{\mathrm{iid}}_{\mathrm{ae}}$, not something it excludes. □

Derived keys can coincide. HMAC outputs for distinct contexts are distinct only with high probability. Two equal key instances with per-object counters starting at 0 give real key-and-nonce reuse. The bound charges this event, at most $K_{\mathrm{key}}(K_{\mathrm{key}} - 1)/2^{257}$ (Fig. 6). Key derivation lowers its probability; it does not make it impossible.

Theorem 25 (Restricted-adaptive composition)**.** *Let N be the number of recipient slots and $M \le \sum_g n_g$ the number of target capsules. For the target epoch, let q be the number of honest object contexts, that is,* Store *calls including the challenge and abandoned creations. Let r be the number of* Open *requests not consisting solely of honest chunk ciphertexts for their context. Let C be the total number of chunk encryptions and D_{dec} the total number of chunk decryptions of ciphertexts that are not honest outputs. Then the number of HMAC evaluations is $Q_{\mathrm{PRF}} \le q + r$, the number of derived-key* instances *(one per distinct context, not necessarily distinct byte strings) is $K_{\mathrm{key}} \le q + r$, and in the game of Definition 21*

$$\mathrm{Adv}(\mathcal{A}) \le L \cdot \Big[\varepsilon_{\mathrm{mu}}(N, M) + \varepsilon_{\mathrm{prf}}(q + r) + \frac{q(q-1)}{2^{257}} + \varepsilon^{\mathrm{iid}}_{\mathrm{ae}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}})\Big], \quad (7)$$

with $L = |\mathcal{P}|$ as in (6)*. More generally, L may be replaced by the size of any set of patterns that contains every pattern a fresh run can produce; under forward-ordered exposure this gives $L = 1$.*

Restricted, not full, adaptive security. Theorem 25 lets the adversary choose exposures adaptively, but it pays the guessing factor L on every term. $L = 1$ only under forward-ordered exposure; it is 970,299 for three 5-of-7 generations with renewed

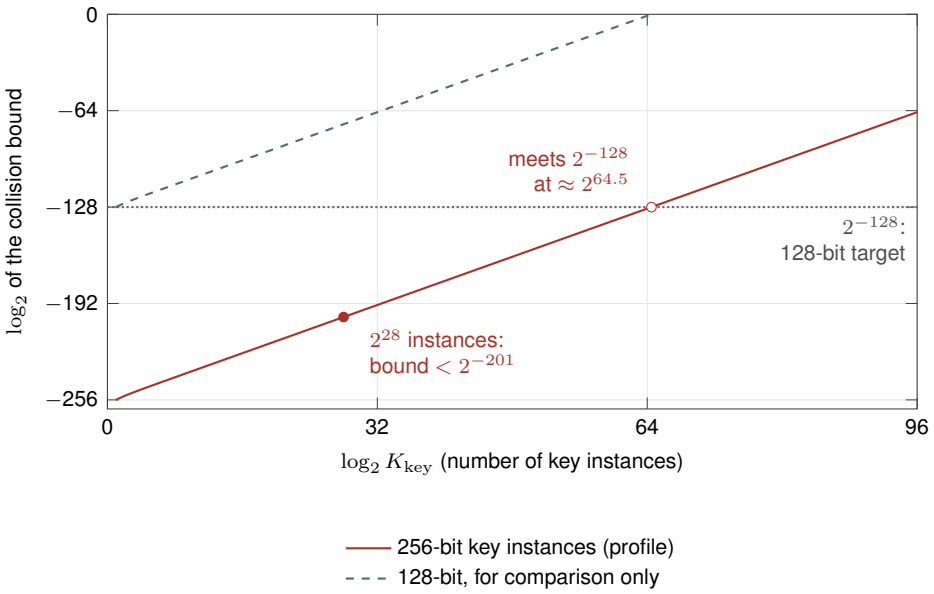


Fig. 6 The derived-key collision bound $K_{\text{key}}(K_{\text{key}}-1)/2^{257}$ of Lemma 24, on a $\log_2$ scale, against the 2^{-128} level of the profile's 128-bit target (dotted). The bound stays below that level until $K_{\text{key}} \approx 2^{64.5}$ key instances. The dashed grey line, for comparison only, is the same birthday bound $K_{\text{key}}(K_{\text{key}}-1)/2^{129}$ for hypothetical 128-bit key instances; it exceeds 2^{-128} for every $K_{\text{key}} \geq 3$. The identifier term $q(q-1)/2^{257}$ of Theorem 25 has the same form, with q in place of K_{key}. This is a bound on the probability of a bad event, not an observed collision frequency.

banks. When L is large, the bound is vacuous. Removing L is open: within this proof approach it would need receiver selective-opening security of the capsule encryption, which HPKE is not known to have (Section 8.2).

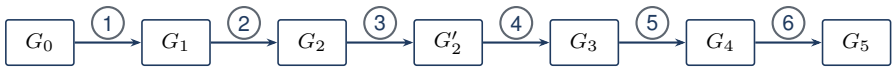
Proof Each hop names its games, reduction and loss; Fig. 7 shows the sequence.

$G_0 \to G_1$ *(guess; exact factor L).* G_1 samples $\Gamma = (\Gamma_g)$ uniformly from $\mathcal{P}$. Its rule: output $\mathcal{A}$'s guess only if the run is fresh and $\Lambda = \Gamma$, and otherwise a fair bit. It stops with a fair bit as soon as a capsule outside Γ is late-exposed, or as soon as the run becomes non-fresh: a target SeedRoute, Open on the challenge, or some $|I_g|$ reaching t_g. Non-freshness is monotone, so stopping early does not change the output distribution, and later reductions never have to answer those queries. Runs with $\Lambda = \Gamma$ never trigger the pattern stop, so their outputs equal those of G_0. Write F for freshness and W for $b' = b$. Then $\Pr_{G_0}[\mathsf{out} = 1] - \frac{1}{2} = \Pr[F \wedge W] - \frac{1}{2}\Pr[F]$. Couple G_1 with a run of G_0 and an independent Γ. Every fresh run has $\Lambda \in \mathcal{P}$, so $\Pr[\Lambda = \Gamma \mid \text{run}] = 1/L$ on fresh runs, and $\Pr_{G_1}[\mathsf{out} = 1] - \frac{1}{2} = \Pr[F \wedge \Lambda{=}\Gamma \wedge W] - \frac{1}{2}\Pr[F \wedge \Lambda{=}\Gamma] = \frac{1}{L}\big(\Pr[F \wedge W] - \frac{1}{2}\Pr[F]\big)$. Hence $\mathrm{Adv}_0 = L \cdot \mathrm{Adv}_1$ exactly, for this $\mathcal{P}$ and for any other pattern set containing every fresh pattern. The equality is exact even if $\mathcal{P}$ contains unreachable patterns; such patterns only enlarge L, so the final bound is looser but still valid.

$G_1 \to G_2$ *(capsules;* $\varepsilon_{\text{mu}}(N, M)$*).* Call a target capsule *eligible* if its index is outside Γ_g and its slot is unexposed at publication. G_2 replaces every eligible capsule by an HPKE encryption of 96 zero bytes under the same key and *info*. The reduction against the multi-user game takes the directory from its challenger, samples Γ, and simulates everything else itself.

Table 9 Counting conventions for Theorem 25, per case of the data interface. "Context" means a new HMAC evaluation and a new key instance.

Case	Counted as
Honest object, one or ℓ chunks	one context in q; ℓ calls in C
Abandoned creation and retry	two contexts in q; the partial and the full encryptions in C; the retry never reuses the old identifier
Open of honest chunks for an honest context	nothing; answered from records
Open with any non-honest chunk	one request in r; at most one new context (none if the context is cached); one call in D_{dec} per chunk attempted, including the failing one
Repeated Open on a cached malicious context	one request in r; no new context
Empty Open	rejected before key derivation; not counted
Same identifier sampled twice by honest Store	the identifier event, $q(q-1)/2^{257}$
Distinct contexts, equal key bytes	the key-instance event inside $\varepsilon_{\mathrm{ae}}^{\mathrm{iid}}$, or $K_{\mathrm{key}}(K_{\mathrm{key}}-1)/2^{257}$ via Lemma 24



$\Pr = \frac{1}{2}$

Hop	Games	Replacement	Loss
1	$G_0 \to G_1$	guess the pattern $\Gamma \in \mathcal{P}$	exact factor: $\mathrm{Adv}_0 = L\,\mathrm{Adv}_1$
2	$G_1 \to G_2$	zero every eligible capsule	$\varepsilon_{\mathrm{mu}}(N, M)$
3	$G_2 \to G_2'$	decouple the target seed	0 (invariant I)
4	$G_2' \to G_3$	HMAC $\to$ random function	$\varepsilon_{\mathrm{prf}}(q{+}r)$
5	$G_3 \to G_4$	stop on an identifier collision	$q(q{-}1)/2^{257}$
6	$G_4 \to G_5$	AEAD real $\to$ ideal	$\varepsilon_{\mathrm{ae}}^{\mathrm{iid}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}})$

queries: $\le M$ challenges, $\le q{+}r$ PRF, C chunk encryptions, D_{dec} chunk decryptions

$$\mathrm{Adv}(\mathcal{A}) \le L \cdot \left[\varepsilon_{\mathrm{mu}} + \varepsilon_{\mathrm{prf}} + q(q{-}1)/2^{257} + \varepsilon_{\mathrm{ae}}^{\mathrm{iid}}\right]$$

forward-ordered exposure: $\mathcal{P} = \{\varnothing\}$, so $L = 1$

Fig. 7 The hybrid sequence of Theorem 25. The numbered hops are listed in the table with their replacement and distinguishing loss; the query counts and the resulting bound appear below the table. The guessing step multiplies the whole accumulated bound by L; under forward-ordered exposure $L = 1$.

It obtains each eligible capsule from the challenge oracle on (real pair, zeros), at most M queries, and encrypts every other capsule itself. It answers Dec through its decryption oracle; capsules it created itself are honest capsules, which the game already excludes from Dec. On Expose(s) it calls Corrupt unless s holds an eligible capsule. In that case the exposure is a late exposure outside Γ, and both G_1 and G_2 stop with a fair bit, so the reduction never makes an inadmissible corruption. Its simulation is G_1 or G_2 according to the challenge bit, with identical stopping rules.

$G_2 \to G_2'$ *(seed decoupling; loss* 0*).* G_2' shares an independent uniform S' in the target generations while data encryption still uses S. The step rests on the following invariant. (I) On every run of G_2 that is fresh and has $\Lambda = \Gamma$, every real target capsule is at an index in

I_g. Proof: a real target capsule is non-eligible. Either its slot was exposed at publication, so its index is touched; or its index lies in $\Gamma_g = \Lambda_g$, so its slot was exposed after publication and the index is again touched. (I) is conditional on $\Lambda = \Gamma$, which is exactly the condition under which G_2 outputs $\mathcal{A}$'s guess. Eligible capsules encrypt zeros. Fix all randomness other than the target coefficient blocks; the data answers then depend on S only. The translation of Theorem 14 moves the shared value by any Δ while preserving commitments, every coordinate at indices in I_g, and hence every real capsule and every other answer. Acceptance is a function of the transcript and the independent Γ, so by the argument of Corollary 5 each accepted transcript has the same probability whatever value is shared. Therefore the joint law of (accepted transcript, S) is the same whether the sharing uses S or S', and other runs output fair bits. So $\Pr[G_2 = 1] = \Pr[G'_2 = 1]$.

$G'_2 \to G_3$ *(PRF;* $\varepsilon_{\mathrm{prf}}(q + r)$*).* In G'_2, S is used only as the HMAC key for target object keys. G_3 replaces $\mathrm{HMAC}(S, \cdot)$ by a random function. The reduction queries its oracle once per distinct context: the q honest contexts, and at most one new context per non-honest Open request, since each request names exactly one context and keys are memoized. That is at most $q + r$ adaptive queries. An Open consisting solely of honest chunks for an honest context needs no query, because the reduction answers it from its records. A request that reuses an honest context, including an honest identifier with the same header, gets the honest key consistently. Counting only q would omit adversarial contexts; counting chunk decryptions would over-count the PRF queries and conflate them with chunk calls.

$G_3 \to G_4$ *(identifiers;* $q(q-1)/2^{257}$*).* G_4 stops with a fair bit if two honest target objects receive the same uniformly random 32-byte identifier, which happens with probability at most $\binom{q}{2} 2^{-256} = q(q-1)/2^{257}$. Otherwise all honest contexts are distinct, so the honest object keys are independent and uniform. Keys derived for adversarial contexts are either an honest key, when the context equals an honest one, or further independent uniform keys. Adversarially chosen identifiers are not sampled and play no part in this probability. Distinct contexts receive independent uniform key instances under the random function, which is exactly the instance distribution of the i.i.d. AE game. Two instances may coincide, and because every object's chunk counter starts at 0, a coincidence is a genuine reuse of key bytes with the same nonce. This event is not excluded here. It is carried by $\varepsilon^{\mathrm{iid}}_{\mathrm{ae}}$, and it is charged explicitly as $K_{\mathrm{key}}(K_{\mathrm{key}} - 1)/2^{257}$ when Lemma 24 converts to a distinct-key bound. The identifier event above concerns equal contexts, which give equal keys by construction; the key-instance event concerns distinct contexts with equal outputs. Identifiers are only relevant within the target epoch, because the context also binds account, configuration and birth epoch. The two games differ only on the collision event, and since the bound uses identifiers alone, it is conservative when headers also differ.

$G_4 \to G_5$ *(AEAD;* $\varepsilon^{\mathrm{iid}}_{\mathrm{ae}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}})$*).* G_5 answers every target-object chunk encryption, including the challenge's, with uniform strings of the correct length, and every chunk decryption of a non-honest ciphertext with $\perp$. The reduction uses the i.i.d. multi-key all-in-one game, assigning one instance index to each distinct context, so at most $K_{\mathrm{key}} \le q + r$ instances, some used only for decryption. Its instances have exactly the distribution of the G_4 object keys, collisions included, so the simulation is perfect. It forwards the C chunk encryptions and the D_{dec} chunk decryptions of non-honest ciphertexts, and answers honest chunks inside an Open from its records. A single Open may contribute several calls to D_{dec}. Nonces are distinct under each key because each chunk index is used once, and the profile's limit of $2^{32} - 1$ chunks keeps the 64-bit index from wrapping. An interrupted creation is abandoned under a fresh identifier, so no key and nonce pair is reused with different plaintext. It answers Open on honest non-challenge ciphertexts from its own record of the stored plaintext. In G_5 the challenge is independent of b, so $\Pr[G_5 = 1] = \frac{1}{2}$.

Normalization. Each hop above bounds $|\Pr[G_i \Rightarrow 1] - \Pr[G_{i+1} \Rightarrow 1]|$ by the distinguishing advantage of its reduction. The reduction runs $\mathcal{A}$ and outputs the simulated game's out bit, and each primitive term is in the distinguishing form of Definition 22, so no factor of 2 arises. The triangle inequality gives $\mathrm{Adv}_1 = |\Pr[G_1 \Rightarrow 1] - \frac{1}{2}| \le \sum_i |\Pr[G_i \Rightarrow 1] - \Pr[G_{i+1} \Rightarrow 1]| + |\Pr[G_5 \Rightarrow 1] - \frac{1}{2}|$, and the last term is 0. The same convention holds in Lemma 23: B_j's distinguishing advantage is exactly $\frac{1}{N}$ times the hybrid gap.

Summing the hops bounds Adv_1, and $\mathrm{Adv}_0 = L \cdot \mathrm{Adv}_1$. Under forward-ordered exposure $\Lambda = \varnothing$ always holds, so the guess is trivial and $L = 1$. □

Corollary 26 (Concrete loss)**.** *With Lemmas 23 and 24,*

$$\mathrm{Adv}(\mathcal{A}) \le L \cdot \Big[N\, M\, \varepsilon_{\mathrm{cca}} + \varepsilon_{\mathrm{prf}}(q+r) + \frac{q(q-1)}{2^{257}} + \frac{K_{\mathrm{key}}(K_{\mathrm{key}}-1)}{2^{257}} + \varepsilon_{\mathrm{ae}}^{\mathrm{dist}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}}) \Big], \qquad K_{\mathrm{key}} \le q + r.$$

The last two terms may be replaced by $\varepsilon_{\mathrm{ae}}^{\mathrm{iid}}(K_{\mathrm{key}}, C, D_{\mathrm{dec}})$ whenever the AE bound used is proved for i.i.d. instances with per-instance nonce respect. The collision term is negligible in practice: for $K_{\mathrm{key}} \le 2^{28}$ it is below 2^{-201}. The guessing-space size is $L = 4, 16, 99, 848$ for a single generation with $(n,t) = (3,2), (5,3), (7,5), (10,7)$, 256 for two $(5,3)$ generations, and 970,299 for three $(7,5)$ generations. L multiplies every term. The bound is affordable for a few target generations and small committees, and vacuous for many refreshes of a long-lived seed unless exposure is forward-ordered or the reachable pattern set is much smaller.

When the bound is useful.

Write $\varepsilon_{\mathrm{base}}$ for the bracket in (7), so that $\mathrm{Adv}(\mathcal{A}) \le L\,\varepsilon_{\mathrm{base}}$. Since the record game's advantage never exceeds $\frac{1}{2}$, the bound says something only if $L\,\varepsilon_{\mathrm{base}} < \frac{1}{2}$. To guarantee $\mathrm{Adv}(\mathcal{A}) \le 2^{-\lambda}$ the primitive terms must satisfy $\varepsilon_{\mathrm{base}} \le 2^{-\lambda}/L$; the primitives must supply $\log_2 L$ bits beyond the target. Table 10 lists L and $\log_2 L$ for the cases used in this paper. The column $\log_2 L$ is the multiplicative loss of this bound in bits, not a security level of the scheme.

Read this way, forward-ordered exposure costs nothing, and one target generation costs at most 6.6 bits for 5-of-7. Each further renewed 5-of-7 generation adds another $\log_2 99 \approx 6.6$ bits: about 20 bits for three generations and 40 for six (Fig. 8). With primitives that leave a comfortable margin, a few target generations are affordable. Many refreshes of a long-lived seed with late exposures are not, unless exposure is forward-ordered. The reused-bank counts (12.0 bits for three generations, 15.7 for six) hold only for the delivery map of Proposition 27; any other map uses the general L.

Proposition 27 (Reachable patterns under reused and renewed banks)**.** *Suppose the G target generations are published in a fixed order, a run may stop after any prefix, all generations use the same n holders, and each holder has one slot used by every generation (a reused bank). Let $\tau_m = \min_{g \le m} t_g$ for $1 \le m \le G$. The patterns produced by fresh runs are exactly the tuples $\Lambda_g = \{x : \ell_x \ge g\}$ over level functions $\ell : [n] \to \{0, \dots, G\}$ whose support $J = \{x : \ell_x \ge 1\}$ satisfies $J = \varnothing$ or $|J| < \tau_{\max_x \ell_x}$.*

Table 10 Guessing factor of Theorem 25 for representative exposure situations (exact values). Reused-bank rows use Proposition 27, which holds only for that delivery map.

Exposure situation	L	$\log_2 L$
Forward-ordered (any policy)	1	0
One 3-of-5 generation	16	4.0
One 5-of-7 generation	99	6.6
Two 3-of-5 generations, renewed banks	256	8.0
Two 3-of-5 generations, reused bank	51	5.7
Three 5-of-7 generations, renewed banks	970,299	19.9
Three 5-of-7 generations, reused bank	3,991	12.0

Their number is

$$L_{\text{reuse}} = 1 + \sum_{m=1}^{G} \sum_{j=1}^{\tau_m - 1} \binom{n}{j} \big(m^j - (m-1)^j\big),$$

and Theorem 25 holds with L_{reuse} in place of L. Under a common threshold t the sum telescopes to $\sum_{j<t} \binom{n}{j} G^j$. If every run publishes all G generations, the count is $\sum_{j<t_{\min}} \binom{n}{j} G^j$ with $t_{\min} = \tau_G$. If instead every generation uses its own disjoint slots (renewed banks), the reachable set is all of $\mathcal{P}$.

Proof With one slot per holder, holder x is characterized by the time of its slot's exposure: never, just before the k-th publication, or after the last. Its index is late-exposed in generation g iff the exposure follows the g-th publication. With ℓ_x the number of generations published before the exposure (or 0 if never exposed), $\Lambda_g = \{x : \ell_x \geq g\}$. The map $\ell \mapsto \Lambda$ is injective, because $\ell_x = \max\{g : x \in \Lambda_g\}$ or 0. Every x with $\ell_x \geq 1$ is touched in every published generation. So when all G generations are published, freshness forces $|\{x : \ell_x \geq 1\}| < t_{\min}$. Conversely, any set J of fewer than $t_{\min}$ holders with any levels in $\{1, \ldots, G\}$ is realized by exposing exactly those slots at the corresponding times, which touches exactly J in every generation, so the run is fresh. Counting by $|J| = j$ gives $\sum_{j<t_{\min}} \binom{n}{j} G^j$ when all generations are published. Now let a run publish the first G' generations. Every level satisfies $\ell_x \leq G'$, and freshness requires $|J| < \tau_{G'}$. A pattern with maximum level $m \geq 1$ is therefore realizable iff $|J| < \tau_{G'}$ for some $G' \geq m$, iff $|J| < \tau_m$, because τ is non-increasing; stopping after the m-th publication realizes it. Counting by maximum level m and support size j, the level vectors on a fixed J with values in $\{1, \ldots, m\}$ and maximum exactly m number $m^j - (m-1)^j$. Adding the empty pattern gives L_{reuse}, and the common-threshold sum telescopes. Disclosures only enlarge the touched sets, so they cannot create new patterns. The theorem allows any covering pattern set. With disjoint slots the exposure times of different generations' slots are independent, so every $\Lambda \in \mathcal{P}$ is realized by exposing exactly its indices after the corresponding publications. □

For three 5-of-7 generations, $L_{\text{reuse}} = 3{,}991$ against $L = 970{,}299$ (Fig. 8); for two 3-of-5 generations, 51 against 256; for $n = 5$ with thresholds $(4, 2, 2)$ and partial publication, 36. The proposition applies only to the reused-bank delivery map: a fixed publication order, the same n holders in every generation, one slot per holder used by every generation, and runs that may stop after any prefix of publications. For general or partially overlapping delivery maps we have no closed form, and the conservative L of (6) applies. Reused banks shrink the guessing loss precisely because exposures

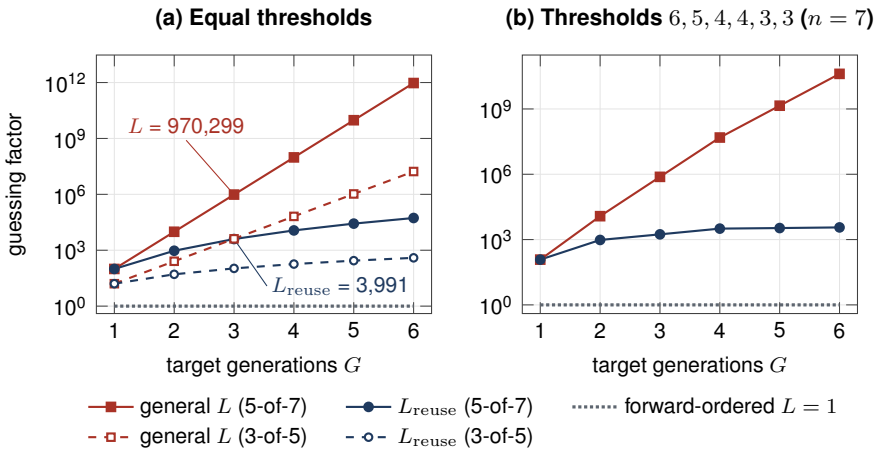


Fig. 8 Guessing factor versus the number of target generations: the general factor L of (6), the exact reused-bank count of Proposition 27, and forward-ordered exposure ($L = 1$). (a) Equal thresholds (5-of-7 and 3-of-5). (b) Thresholds $6, 5, 4, 4, 3, 3$ with $n = 7$ and partial publication. Red squares: general L; navy circles: L_{reuse}; solid lines and filled markers 5-of-7, dashed lines and open markers 3-of-5; grey dotted line: forward-ordered, $L = 1$. All values are exact.

are correlated across generations, although Corollary 16 shows that reuse is worse for secrecy. Renewal restores the full guessing space, so the theorem's loss is larger for the better-protected configuration.

Reused banks are not safer. $L_{\text{reuse}} \ll L$ only means that the *proof* guesses fewer patterns. A reused bank lets one copied root open every later generation (Fig. 5a; Corollary 16), so its actual confidentiality is worse than with renewed banks.

Selective exposure and the separate counts.

If the exposure pattern Γ is declared before the game, Theorem 25 applies with $L = 1$ and with M counting only target capsules outside the declared exposure. The capsule hop then needs only single-key challenges, giving $M\varepsilon_{\text{cca}}$ in place of ε_{mu}, because the declaration tells the reduction which slots are never corrupted. The counts in (7) are kept apart on purpose. One Open evaluates HMAC at most once but decrypts every supplied chunk, so the PRF and key-instance counts are $q + r$, while the AE term counts C chunk encryptions and D_{dec} chunk decryptions. An AE term counted in objects would not suffice, since each object has up to $2^{32} - 1$ chunks under one key.

What full adaptivity would require.

Within the present proof approach, removing L means hiding the contents of capsules whose keys may be revealed later, without knowing in advance which they will be. A receiver selective-opening guarantee for share-distributed messages would suffice. We do not claim that it is necessary, since a different construction or proof could avoid it. Indistinguishability-based versions are known from variants of noncommitting encryption [25, 26], and ordinary IND-CPA or IND-CCA security is not known to imply it [24]. Generalized-selective-decryption techniques [14, 27] can reduce the loss for

structured patterns; we do not claim such an improvement. A programmable-random-oracle DEM built from HPKE's exporter is a plausible smaller architectural change, but we have not proved it, and the profile's ChaCha20 keystream is not programmable. We therefore claim restricted adaptivity only.

8.3 Reference data layer

The artifact contains a reference implementation of the data interface of Definition 21 (Online Resource 1, Fig. S1). It is built only from maintained primitives: HMAC-SHA256 from the Python standard library and ChaCha20-Poly1305 from pyca/cryptography 50.0.2 (OpenSSL 4.0.3). It implements typed deterministic CBOR contexts, memoized per-context keys, chunked encryption with the nonce rule of Table 7, abandonment and retry under a fresh identifier, and per-chunk authentication failures. A guard rejects any reuse of a key-byte and nonce pair within one data-layer instance (scope below). It is a reference, not a production system: it holds objects in memory and does not stream.
Implementation finding: truncation. With one key per object and the chunk index as nonce, per-chunk authentication does not detect an object whose final chunks have been removed, unless the associated data marks the last chunk: in the executed test a four-chunk object with its last two chunks removed is accepted as a 20-byte object without the flag and rejected with it (Online Resource 1, Fig. S2). The profile binds a final-chunk flag, as in the STREAM construction [41]. The decoder computes the flag from each chunk's position. Every chunk must authenticate with $\mathit{final} = 1$ exactly on the last one, and no plaintext is returned unless all chunks verify. So missing, duplicated or early final markers, appended, duplicated or reordered chunks, cross-object chunks and partial failures are all rejected. This is an integrity property of whole objects; it does not change Theorem 25, which concerns confidentiality.

Record validation and the scope of the nonce guard.

Records are validated (exact field set and types, a 32-byte identifier, non-empty chunks of at least tag length) and foreign contexts are rejected before any HMAC evaluation; the key cache and the nonce guard are updated under a lock. A legitimate empty object is stored as one zero-length chunk at index 0, authenticated with $\mathit{final} = 1$, and an Open with no chunks is rejected before key derivation, so removing every chunk of an object never decodes as an empty plaintext. The key-byte and nonce guard is an in-memory set of one data-layer instance, protected by a lock against concurrent threads of that process. It is not shared between processes or service instances and does not survive a restart. A test shows that two instances with forced equal key bytes both encrypt under the same nonce. The guard is a local detection mechanism. The proof does not rely on it: coinciding key instances are accounted for by $\varepsilon_{\mathrm{ae}}^{\mathrm{iid}}$, or by $K_{\mathrm{key}}(K_{\mathrm{key}} - 1)/2^{257}$ through Lemma 24. Online Resource 1 (Table S1) separates what is established from what is not. A deployment that needs global uniqueness could allocate identifiers durably, separate keys per instance, or use a construction whose nonce management does not depend on a process-local set; these are deployment options, not properties of the reference code.